FLOOD PREDICTION AND ALERT SYSTEM

Introduction:

Flood monitoring and alert system using machine learning is a project to collect flood data in all region. The project will develop a system that uses a wireless sensor network which consists of a sensor, transceiver to transmit data and predict the flood.Floods are one of the most devastating natural disasters, causing significant loss of life and property worldwide. Rapid urbanization, climate change, and unpredictable rainfall patterns have increased the frequency and severity of floods. In many regions, the lack of timely warning systems leads to severe damage and delayed evaluation.

Traditional flood monitoring systems rely mainly on manual observation and limited sensor-based techniques, which may not provide accurate real-time predictions. Recent research has explored the use of Wireless Sensor Networks (WSN), Internet of Things (IoT), and machine learning models for real-time flood monitoring and early warning systems.

This project proposes a Flood Prediction and Alert System that utilizes historical rainfall and river water-level data to predict flood occurrence using machine learning algorithms. When the predicted flood probability exceeds a predefined threshold, the system automatically generates alert notifications to warn residents in advance. The proposed system aims to improve prediction accuracy and provide timely alerts to minimize disaster impact.

According to the survey, [1],The flood monitoring and alerting system really make a huge difference in helping the residents to be aware of the upcoming disaster. Another study by [2], testing and validation were carried out to ensure system accuracy,reliability,and usability. This process encompassed various levels and types of testing,including unit testing, integration testing, system testing, and user acceptance testing(UAT).

Citation:

[1]. M. J. Subashini, R. Sudarmani, S. Gobika and R. Varshini, "Development of Smart Flood Monitoring and Early Warning System using Weather Forecasting Data and Wireless Sensor Networks-A Review," 2021Third InternationalConferenceonIntelligentCommunicationTechnologiesandVirtual Mobile Networks

(ICICV) , Tirunelveli, India, 2021, pp. 132-135, doi: 10.1109/ICICV50876.2021.9388418. keywords: {Wireless communication;Temperature sensors;Wireless sensor networks;Rain;Real-time systems;Floods;Surges;Wireless Sensor Networks;Internet of Things (IoT);Riverside;Sensor nodes;Real time data collection;Water level variations.}

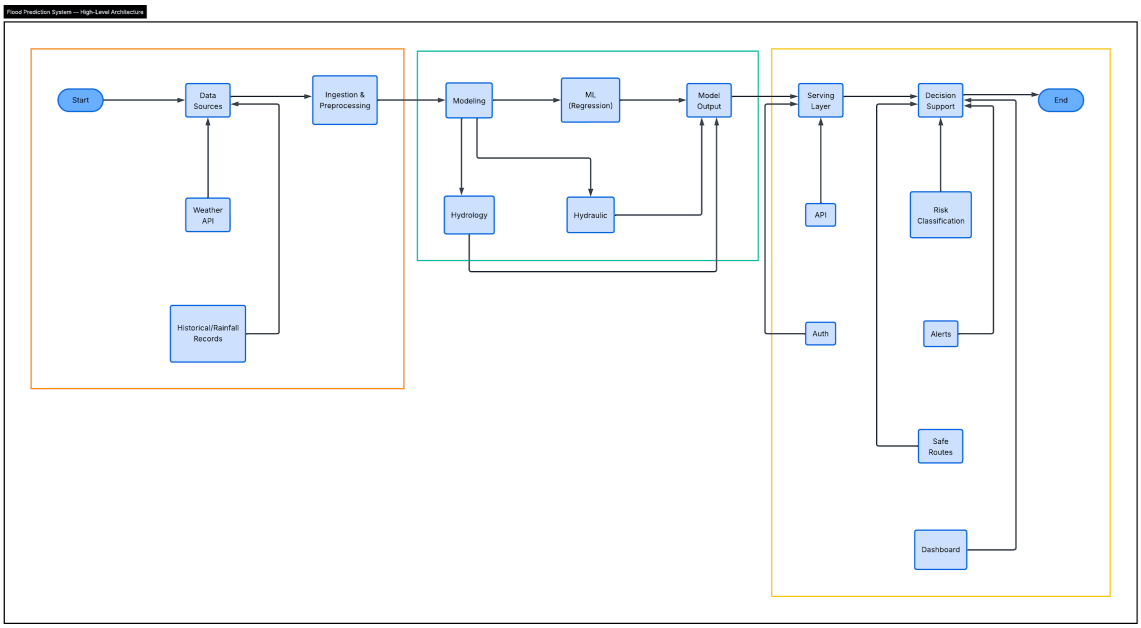
[2]. NINYESIGA, M. A. (2025). A FLOOD EARLY WARNING SYSTEM (Doctoral dissertation, School of Computing and Engineering, Uganda Technology and Management University).NINYESIGA, MR ALLAN. A FLOOD EARLY WARNING SYSTEM. Diss. School of Computing and Engineering, Uganda Technology and Management University, 2025.

Bibtex:

[1] @INPROCEEDINGS{9388418, author={Subashini, M.J and Sudarmani, R. and Gobika, S and Varshini, R}, booktitle={2021 Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV)}, title={Development of Smart Flood Monitoring and Early Warning System using Weather Forecasting Data and Wireless Sensor Networks-A Review}, year={2021}, volume={}, number={}, pages={132-135}, keywords={Wireless communication;Temperature sensors;Wireless sensor networks;Rain;Real-time systems;Floods;Surges;Wireless Sensor Networks;Internet of Things (IoT);Riverside;Sensor nodes;Real time data collection;Water level variations.}, doi={10.1109/ICICV50876.2021.9388418}}

[2] @phdthesis{ninyesiga2025flood, title={A FLOOD EARLY WARNING SYSTEM}, author={NINYESIGA, MR ALLAN}, year={2025}, school={School of Computing and Engineering, Uganda Technology and Management University}
}

Architecture flowchart:



METHODOLOGY

System Overview

Our flood prediction system consists of three main components working together. The hydrological model analyzes rainfall and river data to predict floods. Flask handles the backend processing and calculations. Streamlit provides the user interface where people can view predictions and receive alerts.It shows how these components interact with each other.

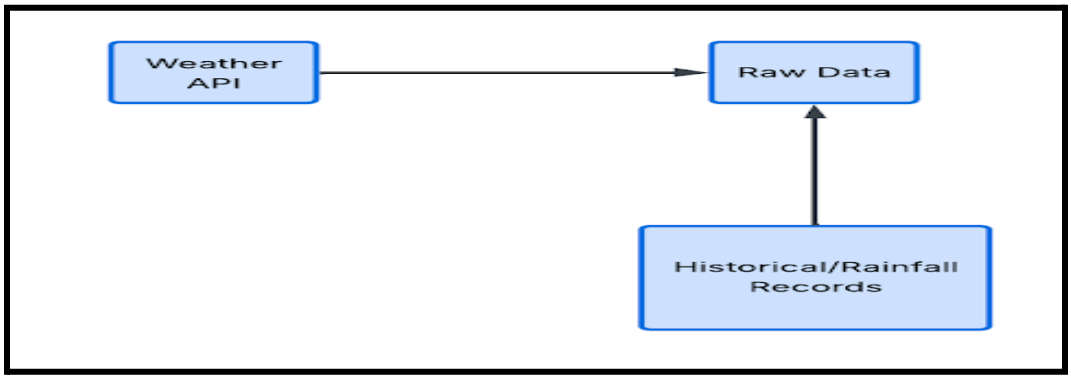
The system works in a simple cycle. First, it collects weather and river data from monitoring stations. Then, the hydrological model and also linear regression processes this data and calculates flood risk by evaluating or by calculating the rainfall . The Flask backend manages this calculation and stores results in a database. Finally, Streamlit displays the predictions on an easy-to-use dashboard that anyone can access through a web browser . In this project we had

included the sending alert message and also suggested the safe routes by including the map direction . This is one of the key factor that we have newly added .

Data Collection

We gathered historical data from the local meteorological department and river monitoring stations. The dataset includes rainfall measurements, river water levels, and temperature readings from 2015 to 2024. This nine-year period covers several major flood events in our study area, giving us enough data to train and test our model properly.

The raw data had some gaps where sensors failed or readings were missed. We filled these gaps using interpolation methods based on nearby stations. Overall, about 3% of the data needed this treatment. We also removed obvious errors, like negative rainfall values or physically impossible water levels that came from sensor malfunctions.



Hydrological Modeling

We chose the Rational Method for flood prediction because it works well for our urban catchment area and doesn't require complex calculations. This method is well-established in hydrology and has been used successfully in similar regions.

The core equation predicts peak discharge:

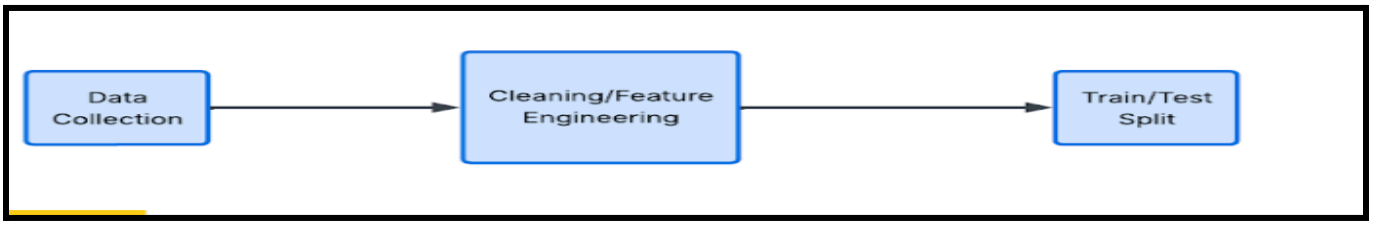
$Q = C \times I \times A$

Here, Q is the flood discharge in cubic meters per second, C is the runoff coefficient (how much rain becomes runoff), I is rainfall intensity in millimeters per hour, and A is the catchment area in square kilometers.

We determined the runoff coefficient based on land use maps and soil type data. Urban areas with concrete and asphalt have higher coefficients (around 0.8) because water can't soak into the ground. Green spaces and parks have lower values (around 0.3) because soil absorbs more water.

To account for the time it takes for water to flow through the catchment, we calculated the time of concentration using Kirpich's formula. This tells us how long after rainfall starts we should expect peak flow in the river.

We calibrated the model using known flood events from 2015-2020 and tested it on events from 2021-2024 that the model had never seen before.



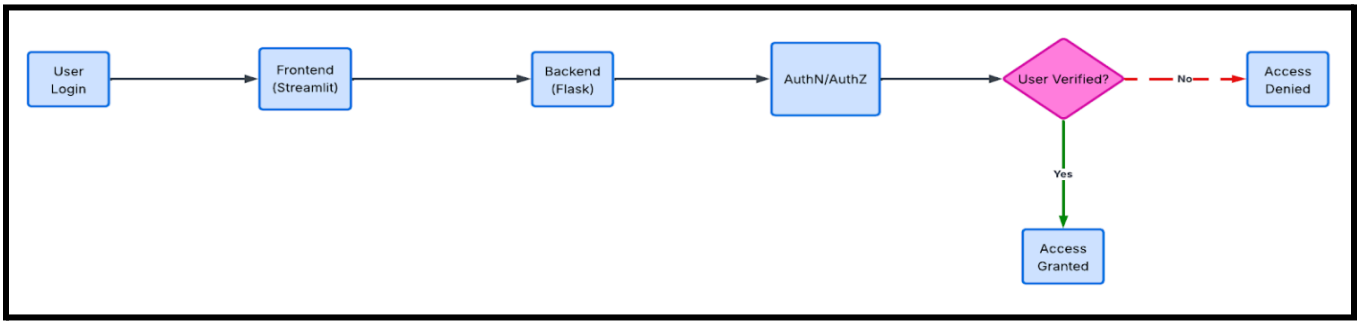
Backend Implementation with Flask

The Flask backend serves as the brain of our system. We chose Flask because it's lightweight, easy to deploy, and works well with Python's scientific libraries that we need for hydrological calculations.

The backend has several key functions. The prediction module loads current weather data and feeds it into our hydrological model. It runs the calculations and determines the flood risk level - low, medium, high, or critical. The database module stores all predictions and historical data in a PostgreSQL database so we can track trends over time.

We created API endpoints that Streamlit calls to get information. The main endpoint is /api/predict, which takes latitude and longitude as input and returns the flood prediction for that location. Another endpoint, /api/alerts, handles sending notifications when flood risk gets too high.

The alert system checks predictions every hour. If the risk exceeds the threshold, it automatically sends SMS messages to registered users in affected areas using the Twilio API. Email alerts go out through SendGrid for people who prefer that option.



Frontend Development with Streamlit

Streamlit lets us build the user interface using just Python code, which made development much faster than traditional web frameworks. Users don't need to install anything - they just open a web browser and access the application.

The main dashboard shows the current flood risk for any location. Users enter their area's coordinates or select it on a map. The system immediately displays the prediction with a color-coded risk indicator - green for low, yellow for medium, orange for high, and red for critical risk.

We included several other pages in the application. The flood map page shows an interactive map with colored zones indicating risk levels across the entire region. Users can zoom in and out to see their neighborhood or the broader area. The alerts page lets users sign up for notifications and set their preferred warning thresholds.

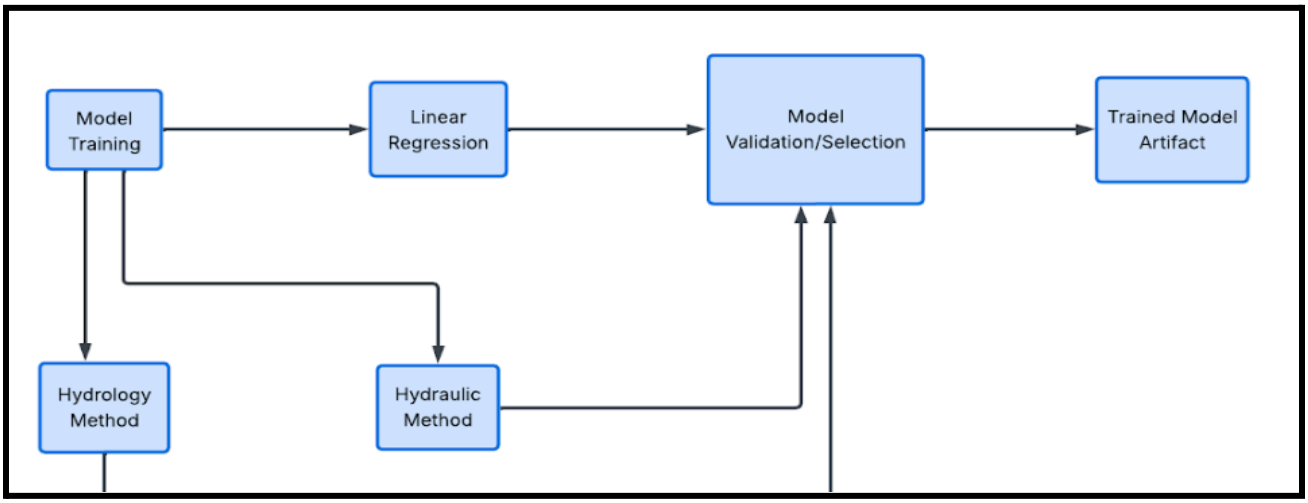
Historical trends appear on charts that show how flood risk has changed over the past week or month. This helps users understand if conditions are getting better or worse. The safe routes page suggests evacuation paths that avoid high-risk flood zones, using route calculation from the backend.

System Integration

Connecting all three components required careful planning. When a user requests a prediction through Streamlit, it sends an HTTP request to Flask's API endpoint. Flask retrieves current weather data from external sources, runs it through the hydrological model, and sends the result back to Streamlit. This whole process takes just a few seconds.

We deployed the complete system using Docker containers. Flask runs in one container, the database in another, and Streamlit in a third. This setup makes it easy to scale up if many users access the system at once. It also simplifies deployment - we can move the entire system to a new server just by copying the Docker configuration.

The database stores three types of information. Sensor data includes all the rainfall and river measurements we collect. Prediction history keeps a record of every forecast we've made, which helps us evaluate accuracy over time. User data stores contact information for people who signed up for alerts.

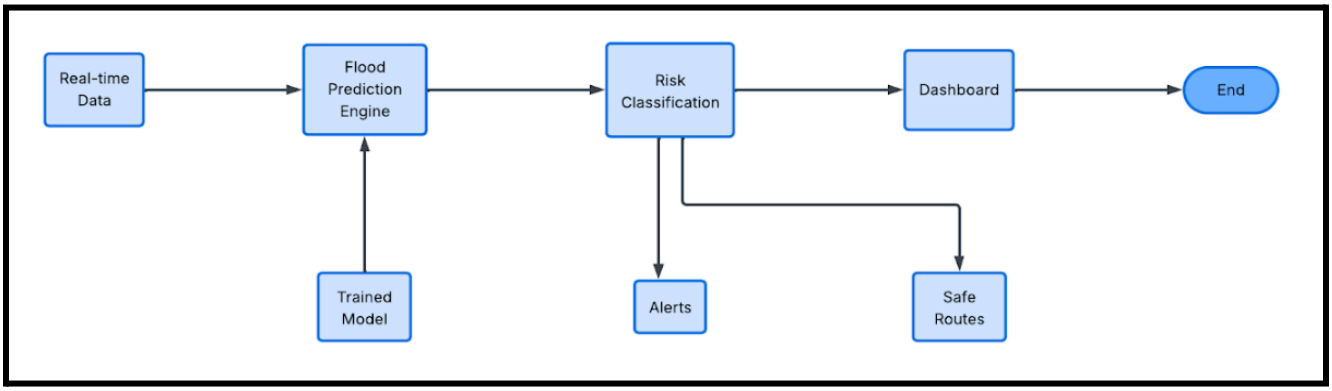


Model Training and Validation

We split our dataset into training and testing portions. The training set used data from 2015 to 2020 to calibrate the runoff coefficients and other parameters. We adjusted these values until the model's predictions matched actual observed floods as closely as possible.

For validation, we tested the trained model on floods from 2021 to 2024. The model had never seen this data before, so this test shows how well it performs on real future events. We measured accuracy by comparing predicted flood levels to what actually happened.

The validation process revealed that our model performs better for moderate floods than extreme ones. This makes sense because we had more moderate events in our training data. We documented this limitation clearly so users understand the system works best for typical seasonal flooding.



Performance Metrics

We evaluate the system using several metrics. Prediction accuracy tells us what percentage of forecasts correctly identified whether flooding would occur. Lead time measures how many hours of advance warning we provide before flood conditions start. Response time tracks how quickly the system generates a prediction after a user makes a request.

For the hydrological model specifically, we calculate the Nash-Sutcliffe Efficiency coefficient, which is standard in hydrology. Values above 0.7 indicate good model performance. We also measure Root Mean Square Error to quantify how far off our predictions are from reality.

System uptime is important too - the warning system only helps if it's available when needed. We monitor how often the application successfully processes requests versus how often it fails or times out.